GHID DE EXECUȚIE

PreFlight

Pas cu pas, de la zero la demo-ul câștigător, în 12 ore.

8 faze, fiecare cu: obiectiv, pași bifabili, comenzi, cod gata de copiat și un test „Go / No-Go” la final.

Ordinea e aleasă ca să elimine riscurile mari în prima oră. Dacă rămâi fără timp, ai mereu un produs care funcționează.

Citește întâi pagina 2 (regulile de aur). Lucrează apoi fază cu fază, fără să sari peste testele Go / No-Go.

Regula nr. 1: demo funcțional > funcții multe

Pregătit pentru hackathonul Adobe • 7 noiembrie 2026

Înainte să începi

Regulile de aur

1. **Riscul întâi.** În prima oră și jumătate verifici doar că piesele se leagă: add-on în Express, funcție în Firebase, apel LLM, C2PA. Fără UI frumos până nu merg astea.
2. **Cod determinist întâi, AI după.** Contrastul și culorile merg fără internet și fără LLM. Dacă AI-ul cade în timpul demo-ului, rămâne jumătate de produs care funcționează.
3. **Commit în fiecare oră** (git) și după fiecare test Go / No-Go reușit. Dacă strici ceva la ora 9, te întorci la ultima variantă bună în 10 secunde.
4. **MUST, apoi SHOULD, apoi COULD.** Nu atingi nimic opțional (PDF Services, Slack, OCR) până nu merge Fix all cap-coadă.
5. **Code freeze la H+10:30.** După ora asta repari doar bug-uri și repeți pitch-ul. Fără funcții noi.
6. **Demo-ul se repetă pe rețeaua din sală**, nu doar acasă. Ai hotspot pe telefon ca rezervă.
7. **Când te blochezi peste 20 de minute** pe ceva, lipește eroarea completă aici, în chat, sau treci la planul B al fazei.

Cronologia (H+ = ore de la start)

Faza	Fereastră	Livrabil	Prioritate
0. Setup și eliminarea riscurilor	H+0:00 – 1:30	Add-on hello world în Express, funcție deployată, LLM răspunde, C2PA citit	MUST
1. Scanare + motor determinist	H+1:30 – 3:30	Texte și culori citite din document, contrast și paletă calculate	MUST
2. Audit AI (Cloud Function)	H+3:30 – 5:30	/audit întoarce JSON valid cu claim-uri și verdict Art. 50(4)	MUST
3. Poarta C2PA la import	H+5:30 – 6:30	Drag & drop: credentials citite, imaginea inserată pe canvas	MUST
4. Panoul UI + scor	H+6:30 – 8:00	Inel de scor, chip-uri, carduri, stări de încărcare	MUST
5. Fix all + re-verificare	H+8:00 – 9:30	Scor 54 → 96 live în Express	MUST
6. Asset-uri, jurnal de audit, extra	H+9:30 – 10:30	VitaBoost A/B/C, Firestore audits, (PDF Services, Slack)	SHOULD / COULD
7. Pitch, video de backup, repetiții	H+10:30 – 12:00	Deck, video de 70 s, 3 repetiții cronometrate	MUST

Dacă lucrezi în echipă

Persoană	Răspunde de
A: Add-on	Fazele 0.3–0.4, 1, 3, 4, 5 (tot ce rulează în Express)
B: Backend și AI	Fazele 0.5–0.7, 2, 6 (Firebase, prompt, Firestore, PDF Services)
C: Produs și pitch	Asset-urile de demo (de la H+2), deck-ul (de la H+6), video de backup, Q&A. Testează constant ce construiesc A și B.

Lucrezi singur? Mergi exact în ordinea fazelor și taie din Faza 6 tot ce e COULD.

FAZA 0 H+0:00 – 1:30

Setup și eliminarea riscurilor

Obiectiv: toate piesele critice comunică între ele într-o formă minimă. Nimic frumos, totul conectat.

0.1 Conturi și unelte (15 min)

- ☐ **Node.js 20 LTS** sau mai nou (`node -v`) și **git**. Pe Windows, CLI-ul de add-on are nevoie de **openssl** în PATH (din Git: `C:\Program Files\Git\usr\bin`).
- ☐ **Cont Adobe Express** (gratuit) în Chrome. Din setările contului activează **Add-on Development** (developer mode).
- ☐ **Proiect Firebase** nou, cu locația Firestore **europe-west1**. Treci pe planul **Blaze** (Cloud Functions îl cer) și setează imediat o **alertă de buget de 5 $** în Google Cloud Billing.
- ☐ **Cheie LLM:** Gemini din Google AI Studio (aistudio.google.com) sau Anthropic Console. **Abonamentul Claude Pro NU include acces API.** Notează numele exact al modelului rapid disponibil (familia Flash / Haiku).
- ☐ **Întreabă organizatorii:** (a) e permis un add-on Express? (b) oferă credențiale pentru Firefly Services sau alte API-uri Adobe? (c) cât durează pitch-ul și demo-ul?

0.2 Repository (2 min)

terminal

```
mkdir preflight-hack && cd preflight-hack && git init
# structura finală:
#   addon/     -> add-on-ul Express
#   backend/   -> Firebase (functions + firestore)
#   demo/      -> asset-uri, brand kit, video de backup
```

0.3 Scheletul add-on-ului (15 min)

terminal

```
npx @adobe/create-ccweb-add-on addon --template swc-javascript-with-document-sandbox
cd addon
npm install
npm run build
npm run start          # serverul local pornește pe https://localhost:5241
```

- ☐ Deschide o dată `https://localhost:5241` în Chrome și acceptă certificatul local, dacă browserul îl blochează.
- ☐ În Express: creează un design gol → panoul **Add-ons** → **Your add-ons** → activează testarea add-on-urilor locale → introdu `https://localhost:5241` → **Connect**. Denumirile din meniu se pot schimba; dacă nu găsești opțiunea, urmează pagina Quickstart din documentația Adobe Express Add-ons.
- ☐ Apasă butonul din template (creează un dreptunghi). Dacă apare pe canvas, **UI-ul și Document Sandbox comunică**. Ăsta e cel mai mare risc tehnic, deci ai trecut de el.
- ☐ Uită-te la fișiere: `src/manifest.json`, `src/index.html`, `src/ui/index.js` (iframe) și `src/sandbox/code.js` (document). Dacă template-ul are alte nume, adaptează căile din acest ghid.

0.4 Backend Firebase (20 min)

terminal

```
npm i -g firebase-tools
firebase login
mkdir ../backend && cd ../backend
firebase init functions firestore   # proiectul tău, JavaScript, fără ESLint, instalează dependențele
cd functions
npm i @google/genai                  # sau: npm i @anthropic-ai/sdk
firebase functions:secrets:set GEMINI_API_KEY      # lipești cheia când ți-o cere
```

backend/functions/index.js (temporar)

```
const { onRequest } = require("firebase-functions/v2/https");

exports.ping = onRequest({ region: "europe-west1", cors: true }, (req, res) => {
  res.json({ ok: true, t: Date.now() });
});
```

terminal

```
firebase deploy --only functions      # copiază URL-ul afișat pentru ping
```

0.5 Testează apelul din add-on (10 min)

addon/src/ui/index.js

```
// în src/ui/index.js, după addOnUISdk.ready:
const r = await fetch("https://URL-UL-TAU-PING");
console.log("PING", await r.json());     // vezi în DevTools (F12) -> Console
```

Capcană

Nu apela emulatorul local (`http://127.0.0.1:5001`) din add-on. Add-on-ul rulează pe https, iar browserul blochează apelurile http (mixed content). Folosește funcția deployată.

0.6 Testează LLM-ul (10 min)

Înlocuiește temporar ping cu un apel minim la model (codul complet e în Faza 2). E suficient să primești un răspuns la „Spune OK”. Dacă primești 429 sau 403, rezolvă acum: cheie, facturare, nume de model.

0.7 Testează ipoteza C2PA (10 min, în paralel)

- ☐ Generează în **firefly.adobe.com** (sau cu „Generate image” din Express): *„photorealistic doctor in white coat holding a supplement bottle, studio light”*. Descarc-o.
- ☐ Încarc-o pe **contentcredentials.org/verify**. Trebuie să vezi Content Credentials, cu generare AI.
- ☐ Pune imaginea într-un design Express, exportă pagina ca PNG și verifică exportul pe același site. **Notează rezultatul.** Oricare ar fi, arhitectura rămâne valabilă: citim C2PA la import. Dacă exportul păstrează sau adaugă credentials, ai un argument în plus pentru pitch.

Go / No-Go

- ☐ Butonul din add-on modifică documentul în Express.
- ☐ Add-on-ul primește JSON de la funcția deployată (fără eroare CORS).
- ☐ Modelul LLM răspunde printr-o funcție deployată.
- ☐ Știi dacă imaginea Firefly are credentials (și ce se întâmplă la export).

Dacă NU trece: dacă add-on-ul nu se conectează nici la H+1:30, treci pe **planul B**: o aplicație web (Firebase Hosting) cu upload de PNG. Faza 1 rulează atunci pe imagine (fără Document Sandbox), iar fix-urile se desenează pe canvas. Restul ghidului rămâne la fel.

FAZA 1 H+1:30 – 3:30

Scanare + motor determinist

Obiectiv: citești textele, culorile și pozițiile din document și calculezi contrastul WCAG și abaterile de la paletă, fără AI.

1.1 Document Sandbox: citire și fix-uri (tot fișierul)

Fișierul conține tot ce trebuie pentru Fază 5. Îl scrii o dată, acum.

addon/src/sandbox/code.js

```
import addOnSandboxSdk from "add-on-sdk-document-sandbox";
import { editor, colorUtils } from "express-document-sdk";

const { runtime } = addOnSandboxSdk.instance;
const nodesById = new Map();

function walk(node, out) {                      // parcurge tot arborele paginii
  for (const child of node.allChildren) {
    out.push(child);
    if (child.allChildren) walk(child, out);
  }
}
const hex = (c) => colorUtils.toHex(c).slice(0, 7);   // toHex dă #RRGGBBAA
const box = (b) => ({ x: b.x, y: b.y, w: b.width, h: b.height });

function scanPage() {
  const board = editor.context.insertionParent;   // artboard-ul paginii curente
  const all = []; walk(board, all);
  nodesById.clear();
  const texts = [], shapes = [];
  for (const n of all) {
    try {
      if (n.type === "Text") {
        const st = n.fullContent.characterStyleRanges[0];
        nodesById.set(n.id, n);
        texts.push({ id: n.id, text: n.fullContent.text, color: hex(st.color),
                     fontSize: st.fontSize, bounds: box(n.boundsInParent) });
      } else if (n.fill && n.fill.color) {
        nodesById.set(n.id, n);
        shapes.push({ id: n.id, type: n.type, fill: hex(n.fill.color), bounds: box(n.boundsInParent) });
      }
    } catch (e) { /* nod care nu poate fi citit: îl ignorăm */ }
  }
  return { page: { width: board.width, height: board.height }, texts, shapes };
}

function setTextColor(id, h) {
  const n = nodesById.get(id); if (!n) return false;
  n.fullContent.applyCharacterStyles({ color: colorUtils.fromHex(h) }); return true;
}
function setFill(id, h) {
  const n = nodesById.get(id); if (!n) return false;
  n.fill = editor.makeColorFill(colorUtils.fromHex(h)); return true;
}
function replaceText(id, txt) {
  const n = nodesById.get(id); if (!n) return false;
  n.fullContent.text = txt; return true;
}
// eticheta "Imagine generată cu AI" (dreapta-jos) sau disclaimerul (stânga-jos)
function addLabel(text, { corner = "br", size = 22, color = "#FFFFFF", bg = "#0F172A", pad = 24 } = {}) {
  const board = editor.context.insertionParent;
  const rect = bg ? editor.createRectangle() : null;
  if (rect) board.children.append(rect);          // întâi fundalul, apoi textul deasupra
  const t = editor.createText(text);
  board.children.append(t);
  t.fullContent.applyCharacterStyles({ fontSize: size, color: colorUtils.fromHex(color) });
```

```
  const b = t.boundsLocal, right = corner === "br";
  const x = right ? board.width - pad : pad, y = board.height - pad;
  if (rect) {
    rect.width = b.width + 20; rect.height = b.height + 12;
    rect.fill = editor.makeColorFill(colorUtils.fromHex(bg));
    rect.setPositionInParent({ x, y }, { x: right ? rect.width : 0, y: rect.height });
  }
  const dx = rect ? (right ? -10 : 10) : 0, dy = rect ? -6 : 0;
  t.setPositionInParent({ x: x + dx, y: y + dy }, { x: right ? b.x + b.width : b.x, y: b.y + b.height });
  return t.id;
}

runtime.exposeApi({ scanPage, setTextColor, setFill, replaceText, addLabel });
```

Atenție

- `boundsInParent` e relativ la părinte. **Nu grupa textele și butoanele** în asset-ul de demo, ca să rămână copii direcți ai artboard-ului.
- Dacă `scanPage()` întoarce 0 texte: loghează `n.type` pentru toate nodurile ca să vezi denumirile reale. Fallback: lucrezi pe `editor.context.selection`.
- Un buton din biblioteca de forme Express poate fi un grup. În demo, desenează CTA-ul ca dreptunghi simplu cu text deasupra.

1.2 Motorul determinist (contrast, paletă, disclaimer, scor)

addon/src/ui/engine.js

```
// ---------- culori ----------
export const hexToRgb = (h) => [1, 3, 5].map((i) => parseInt(h.slice(i, i + 2), 16));
const lin = (c) => { c /= 255; return c <= 0.03928 ? c / 12.92 : ((c + 0.055) / 1.055) ** 2.4; };
export const lum = ([r, g, b]) => 0.2126 * lin(r) + 0.7152 * lin(g) + 0.0722 * lin(b);
export const contrast = (a, b) => { const [x, y] = [lum(a), lum(b)].sort((p, q) => q - p);
                                    return (x + 0.05) / (y + 0.05); };
const INK = [15, 23, 42], WHITE = [255, 255, 255];
export const bestTextOn = (bg) => (contrast(INK, bg) >= contrast(WHITE, bg) ? "#0F172A" : "#FFFFFF");
const toHex = (rgb) => "#" + rgb.map((v) => Math.round(v).toString(16).padStart(2, "0")).join("").toUpperCase();

function rgbToLab([r, g, b]) {                    // sRGB -> CIE Lab (D65)
  [r, g, b] = [r, g, b].map(lin);
  let x = (r * 0.4124 + g * 0.3576 + b * 0.1805) / 0.95047, y = r * 0.2126 + g * 0.7152 + b * 0.0722,
      z = (r * 0.0193 + g * 0.1192 + b * 0.9505) / 1.08883;
  const f = (t) => (t > 0.008856 ? Math.cbrt(t) : 7.787 * t + 16 / 116);
  [x, y, z] = [f(x), f(y), f(z)];
  return [116 * y - 16, 500 * (x - y), 200 * (y - z)];
}
const deltaE = (a, b) => Math.hypot(...rgbToLab(a).map((v, i) => v - rgbToLab(b)[i]));
const chroma = (rgb) => { const [, a, b] = rgbToLab(rgb); return Math.hypot(a, b); };

// ---------- 1. contrast WCAG pe pixeli reali ----------
export async function checkContrast(pngBlob, scan) {
  const bmp = await createImageBitmap(pngBlob);
  const cv = new OffscreenCanvas(bmp.width, bmp.height);
  const ctx = cv.getContext("2d", { willReadFrequently: true });
  ctx.drawImage(bmp, 0, 0);
  const sx = bmp.width / scan.page.width, sy = bmp.height / scan.page.height;
  const issues = [];
  for (const t of scan.texts) {
    const x = Math.max(0, Math.floor(t.bounds.x * sx)), y = Math.max(0, Math.floor(t.bounds.y * sy));
    const w = Math.max(1, Math.min(bmp.width - x, Math.ceil(t.bounds.w * sx)));
    const h = Math.max(1, Math.min(bmp.height - y, Math.ceil(t.bounds.h * sy)));
    const px = ctx.getImageData(x, y, w, h).data, fg = hexToRgb(t.color);
    const R = [], G = [], B = [];
    for (let i = 0; i < px.length; i += 12) {     // 1 din 3 pixeli
      const d = Math.abs(px[i] - fg[0]) + Math.abs(px[i + 1] - fg[1]) + Math.abs(px[i + 2] - fg[2]);
      if (d > 100) { R.push(px[i]); G.push(px[i + 1]); B.push(px[i + 2]); }   // doar fundal
    }
    if (R.length < 20) continue;
```

```
    const med = (a) => a.sort((p, q) => p - q)[a.length >> 1];
    const bg = [med(R), med(G), med(B)];
    const ratio = contrast(fg, bg), min = t.fontSize >= 24 ? 3 : 4.5;
    if (ratio < min) issues.push({
      axis: "accessibility", severity: "error", nodeId: t.id, quote: t.text,
      title: `Contrast ${ratio.toFixed(1)}:1 (minim ${min}:1)`,
      explanation: `Textul „${t.text.slice(0, 40)}” e greu de citit pe fundalul ${toHex(bg)}.`,
      legalBasis: "WCAG 2.2, criteriul 1.4.3",
      fix: { kind: "textColor", nodeId: t.id, hex: bestTextOn(bg) },
    });
  }
  return issues;
}

// ---------- 2. culori în afara paletei (doar forme colorate, ignorăm neutrele) ----------
export function checkPalette(scan, kit) {
  const pal = kit.palette.map(hexToRgb), issues = [];
  for (const s of scan.shapes) {
    const c = hexToRgb(s.fill);
    if (chroma(c) < 12) continue;                  // gri/alb/negru: nu sunt "culori de brand"
    const dists = pal.map((p) => deltaE(c, p)), dmin = Math.min(...dists);
    if (dmin <= 20) continue;
    const target = pal[dists.indexOf(dmin)], targetHex = toHex(target);
    const inside = scan.texts.filter((t) => t.bounds.x >= s.bounds.x && t.bounds.y >= s.bounds.y &&
        t.bounds.x + t.bounds.w <= s.bounds.x + s.bounds.w + 1 &&
            t.bounds.y + t.bounds.h <= s.bounds.y + s.bounds.h + 1);
    const textFixes = inside.filter((t) => contrast(hexToRgb(t.color), target) < 4.5)
                            .map((t) => ({ nodeId: t.id, hex: bestTextOn(target) }));
    issues.push({ axis: "brand", severity: "warning", nodeId: s.id,
      title: `Culoare ${s.fill} în afara paletei`, explanation: `ΔE ${dmin.toFixed(0)} față de ${targetHex}.`,
      legalBasis: "Ghid de brand", fix: { kind: "fill", nodeId: s.id, hex: targetHex, textFixes } });
  }
  return issues;
}

// ---------- 3. disclaimer obligatoriu din ghid ----------
const norm = (s) => (s || "").toLowerCase().normalize("NFD").replace(/[\u0300-\u036f]/g, "").replace(/\s+/g,
    " ").trim();
export function checkDisclaimer(scan, kit) {
  if (!kit.requiredDisclaimer) return [];
  const ok = scan.texts.some((t) => norm(t.text).includes(norm(kit.requiredDisclaimer).slice(0, 40)));
  return ok ? [] : [{ axis: "brand", severity: "warning", title: "Lipsește disclaimerul obligatoriu",
    explanation: kit.requiredDisclaimer, legalBasis: "Ghid de brand", fix: { kind: "disclaimer" } }];
}

// ---------- 4. scor ----------
const WEIGHTS = { brand: 0.25, accessibility: 0.25, legal: 0.30, ai_act: 0.20 };
export function score(issues) {
  const axes = {};
  for (const a in WEIGHTS) {
    const open = issues.filter((i) => i.axis === a && !i.fixed);
    const e = open.filter((i) => i.severity === "error").length,
        w = open.filter((i) => i.severity === "warning").length;
    axes[a] = Math.max(0, 100 - 40 * e - 15 * w);
  }
  const total = Math.round(Object.keys(WEIGHTS).reduce((s, a) => s + axes[a] * WEIGHTS[a], 0));
  return { total, axes };
}
```

1.3 Testează motorul (temporar, din consolă)

addon/src/ui/index.js (test)

```
import { checkContrast, checkPalette, checkDisclaimer, score } from "./engine.js";
import KIT from "./brandkit.json";   // brand kit-ul VitaBoost din dosar, secțiunea 8

const sandbox = await addOnUISdk.instance.runtime.apiProxy("documentSandbox");
const scan = await sandbox.scanPage();
const [r] = await addOnUISdk.app.document.createRenditions(
  { range: addOnUISdk.constants.Range.currentPage, format: addOnUISdk.constants.RenditionFormat.png },
  addOnUISdk.constants.RenditionIntent.preview);
const det = [...await checkContrast(r.blob, scan), ...checkPalette(scan, KIT), ...checkDisclaimer(scan, KIT)];
console.log(scan, det, score(det));
```

Go / No-Go

- ☐ În consolă vezi toate textele VitaBoost cu culoare și poziție.
- ☐ Titlul gri apare cu contrast ~1,5:1, CTA-ul #E11D48 apare în afara paletei, iar lipsa disclaimerului e detectată.
- ☐ `sandbox.setTextColor(id, "#0F172A")` schimbă culoarea în document.

Dacă NU trece: dacă pozițiile sunt greșite, verifică dacă textul e într-un grup și scoate-l din grup. Dacă tot nu merge, calculează contrastul pe toată pagina pentru textul selectat (`editor.context.selection`).

FAZA 2 H+3:30 – 5:30

Audit AI în Cloud Function

Obiectiv: un endpoint /audit care primește imaginea și textele și întoarce JSON validat: claim-uri, ton, verdict Art. 50(4). Are protecție la halucinații și cache.

2.1 Prompt-ul de sistem

SYSTEM_PROMPT

```
Ești PreFlight, auditor de conformitate pentru reclame publicate în România și UE.
Primești: imaginea randată a reclamei, lista textelor din design (id + text), brand kit-ul
și informația de proveniență (C2PA) a imaginilor.

SARCINI
1. LEGAL (axis "legal"), analizând DOAR textele primite:
   - proprietăți de prevenire/tratare/vindecare a bolilor atribuite unui aliment sau
     supliment -> severity "error", legalBasis "Reg. (UE) 1169/2011 art. 7(3)"
   - mențiuni de sănătate neautorizate ("întărește imunitatea", "detoxifiază") ->
     "warning", "Reg. (CE) 1924/2006"
   - superlative absolute sau afirmații nedovedibile ("cel mai bun", "nr. 1", "garantat")
     -> "warning", "Legea 158/2008"
   - reducere procentuală fără preț de referință -> "warning", "Dir. 98/6/CE art. 6a",
     needsHumanInput = true, suggestedText = null
2. BRAND (axis "brand"): abateri CLARE de la brandKit.tone. Nu raporta lucruri minore.
3. aiDisclosure, analizând IMAGINEA:
   photorealistic = arată ca o fotografie reală (nu ilustrație, desen, 3D stilizat)
   depictsRealPersonLike = conține persoane, locuri sau evenimente care pot fi luate drept reale
   disclosureRequired = (provenance.aiGenerated SAU imaginea pare generată cu AI)
                        ȘI photorealistic ȘI depictsRealPersonLike
   reasoning = o propoziție în română

REGULI STRICTE
- "quote" e copiat EXACT dintr-un text primit. Nu inventa și nu parafraza.
- "nodeId" e id-ul textului din care e citatul.
- NU raporta contrast, culori, fonturi sau disclaimere: le verifică alt modul.
- Dacă nu ești sigur, folosește "warning", nu "error". Fără probleme -> issues: [].
- "explanation": maxim 20 de cuvinte, în română, pentru un marketer, nu pentru un jurist.
- "suggestedText": rescriere în română, în tonul brandului, fără problema semnalată,
  cel mult la fel de lungă ca originalul.
- toneScore: 0-100, cât de bine respectă textele tonul brandului.
```

2.2 Funcția completă

backend/functions/index.js

```
const { onRequest } = require("firebase-functions/v2/https");
const { defineSecret } = require("firebase-functions/params");
const { initializeApp } = require("firebase-admin/app");
const { getFirestore, FieldValue } = require("firebase-admin/firestore");
const { GoogleGenAI, Type } = require("@google/genai");
const crypto = require("crypto");

initializeApp();
```

```
const db = getFirestore();
const GEMINI_API_KEY = defineSecret("GEMINI_API_KEY");
const MODEL = "gemini-2.5-flash";   // <- pune numele exact al celui mai nou model Flash din AI Studio
const SYSTEM_PROMPT = `...textul din 2.1...`;

const S = Type;
const SCHEMA = {
  type: S.OBJECT,
  properties: {
    issues: { type: S.ARRAY, items: { type: S.OBJECT, properties: {
      axis: { type: S.STRING, enum: ["brand", "legal"] },
      severity: { type: S.STRING, enum: ["error", "warning", "info"] },
      nodeId: { type: S.STRING, nullable: true },
      quote: { type: S.STRING },
      title: { type: S.STRING },
      explanation: { type: S.STRING },
      legalBasis: { type: S.STRING },
      suggestedText: { type: S.STRING, nullable: true },
      needsHumanInput: { type: S.BOOLEAN },
    }, required: ["axis", "severity", "quote", "title", "explanation"] } },
    aiDisclosure: { type: S.OBJECT, properties: {
      photorealistic: { type: S.BOOLEAN }, depictsRealPersonLike: { type: S.BOOLEAN },
      disclosureRequired: { type: S.BOOLEAN }, reasoning: { type: S.STRING },
    }, required: ["photorealistic", "depictsRealPersonLike", "disclosureRequired", "reasoning"] },
    toneScore: { type: S.INTEGER },
  },
  required: ["issues", "aiDisclosure", "toneScore"],
};

const norm = (s) => (s || "").toLowerCase().replace(/\s+/g, " ").trim();

exports.audit = onRequest(
  { region: "europe-west1", cors: true, secrets: [GEMINI_API_KEY], timeoutSeconds: 60, memory: "512MiB" },
  async (req, res) => {
    try {
      if (req.method !== "POST") return res.status(405).send("POST only");
      const { imageBase64, texts = [], brandKit = {}, provenance = {} } = req.body || {};
      if (!imageBase64) return res.status(400).json({ error: "imageBase64 lipsă" });

      // cache: aceeași imagine + aceeași proveniență -> același rezultat, instant
      const key = crypto.createHash("sha256")
        .update(imageBase64 + JSON.stringify(provenance) + (brandKit.name || "")).digest("hex");
      const hit = await db.collection("cache").doc(key).get();
      if (hit.exists) return res.json({ ...hit.data().result, cached: true });

      const ai = new GoogleGenAI({ apiKey: GEMINI_API_KEY.value() });
      const resp = await ai.models.generateContent({
        model: MODEL,
        contents: [{ role: "user", parts: [
          { inlineData: { mimeType: "image/jpeg", data: imageBase64 } },
          { text: JSON.stringify({ texts: texts.map((t) => ({ id: t.id, text: t.text })), brandKit,
              provenance }) },
        ] }],
        config: { systemInstruction: SYSTEM_PROMPT, responseMimeType: "application/json",
                  responseSchema: SCHEMA, temperature: 0.2 },
      });
      const out = JSON.parse(resp.text);

      // anti-halucinație: păstrăm doar problemele al căror citat EXISTĂ în design
      out.issues = (out.issues || []).filter((i) => {
        const t = texts.find((t) => norm(t.text).includes(norm(i.quote)));
        if (t) i.nodeId = t.id;
        return !!t;
      });
      await db.collection("cache").doc(key).set({ result: out, at: FieldValue.serverTimestamp() });
      res.json(out);
    } catch (e) {
      console.error(e);
      res.status(500).json({ error: String(e.message || e) });
    }
  }
);
```

Folosești Claude în loc de Gemini? Înlocuiește blocul `generateContent` cu un apel `messages.create` din `@anthropic-ai/sdk` (imagine ca bloc `image/base64`, modelul Haiku). Cere JSON prin tool use cu `input_schema` = schema de mai sus. Restul funcției rămâne la fel.

2.3 Reguli Firestore (clientul nu scrie direct)

backend/firestore.rules

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /{document=**} { allow read, write: if false; }   // doar funcțiile (Admin SDK) au acces
  }
}
```

terminal

```
firebase deploy --only functions,firestore:rules
# test rapid cu o imagine locală:
cat > /tmp/req.json <<EOF
{ "imageBase64": "$(base64 -w0 demo/vitaboost.jpg)",
  "texts": [{ "id": "t1", "text": "Vindecă oboseala în 7 zile!" }],
  "brandKit": { "tone": "factual" }, "provenance": { "aiGenerated": true } }
EOF
# pe macOS: base64 -i demo/vitaboost.jpg
curl -s -X POST "https://URL-AUDIT" -H "Content-Type: application/json" -d @/tmp/req.json
```

Go / No-Go

- ☐ /audit întoarce JSON valid. „Vindecă oboseala” apare ca **error** cu Reg. 1169/2011.
- ☐ Pe imaginea cu medicul fotorealist: `disclosureRequired: true`.
- ☐ Al doilea apel identic răspunde în < 1 s, cu `cached: true`.
- ☐ Răspunsul fără cache vine în maximum ~5 secunde.

Dacă NU trece: dacă modelul nu respectă schema, scade temperatura la 0, simplifică schema (scoate toneScore) și loghează `resp.text`. Dacă e lent, micșorează imaginea la 768 px și schimbă modelul cu unul mai rapid.

FAZA 3 H+5:30 – 6:30

Poarta C2PA la import

Obiectiv: utilizatorul trage o imagine în panou, PreFlight citește Content Credentials în browser și abia apoi o inserează pe canvas.

terminal

```
cd addon && npm i @contentauth/c2pa-web
```

addon/src/ui/provenance.js

```
import addOnUISdk from "https://new.express.adobe.com/static/add-on-sdk/sdk.js";
import { createC2pa } from "@contentauth/c2pa-web/inline";   // WASM inclus în bundle, fără hosting separat

const c2paReady = createC2pa();
export const provenance = { hasCredentials: false, aiGenerated: false, generator: null, known: false };

export async function importWithProvenance(file) {
  const c2pa = await c2paReady;
  let store = null;
  try {
    const reader = await c2pa.reader.fromBlob(file.type, file);
    if (reader) { store = await reader.manifestStore(); await reader.free(); }
  } catch (e) {
    console.info("Fără Content Credentials:", e?.message);   // normal pentru imagini fără manifest
  }
  const s = store ? JSON.stringify(store) : "";
  Object.assign(provenance, {
    hasCredentials: !!store,
    // IPTC digitalSourceType pentru conținut generat de AI (acoperă și compositeWith...)
    aiGenerated: /trainedAlgorithmicMedia/i.test(s),
    generator: (s.match(/"claim_generator"\s*:\s*"([^"]+)"/) || [])[1] || null,
    known: true,
  });
  await addOnUISdk.app.document.addImage(file);              // abia acum intră pe canvas
```

```
  return { ...provenance };
}
```

addon/src/index.html (fragment)

```
<div id="drop" class="drop">Trage aici o imagine <small>sau</small>
  <label class="link">alege fișier<input id="file" type="file" accept="image/png,image/jpeg" hidden></label>
</div>
<div id="prov" class="prov"></div>
```

addon/src/ui/index.js (fragment)

```
const drop = document.getElementById("drop");
const onFile = async (f) => {
  const p = await importWithProvenance(f);
  document.getElementById("prov").innerHTML = p.hasCredentials
    ? `Content Credentials găsite${p.aiGenerated ? " • <b>generată cu AI</b>" : ""} (${p.generator ?? "?"})`
    : "Fără Content Credentials: sursa va fi verificată vizual.";
};
drop.addEventListener("dragover", (e) => { e.preventDefault(); drop.classList.add("over"); });
drop.addEventListener("dragleave", () => drop.classList.remove("over"));
drop.addEventListener("drop", (e) => { e.preventDefault(); drop.classList.remove("over");
    onFile(e.dataTransfer.files[0]); });
document.getElementById("file").addEventListener("change", (e) => onFile(e.target.files[0]));
```

Go / No-Go

- ☐ Imaginea Firefly: „Content Credentials găsite • generată cu AI” și apare pe canvas.
- ☐ O poză normală, fără manifest, nu blochează nimic. Mesajul e „Fără Content Credentials”.

Dacă NU trece: dacă pachetul nu se compilează în webpack: (1) încearcă importul cu WASM separat din documentația c2pa-web; (2) mută citirea pe server cu `@contentauth/c2pa-node` într-o funcție /provenance (trimiți fișierul ca base64). Fluxul pentru utilizator rămâne identic.

FAZA 4 H+6:30 – 8:00

Panoul UI + scor

Obiectiv: panoul din dosar (secțiunea 7): inel de scor, 4 chip-uri, carduri de probleme, rezultate deterministe instant și AI progresiv.

4.1 Fluxul principal

addon/src/ui/index.js

```
import { checkContrast, checkPalette, checkDisclaimer, score } from "./engine.js";
import { provenance } from "./provenance.js";
import KIT from "./brandkit.json";
const API_AUDIT = "https://URL-AUDIT";
let sandbox, scan, issues = [], scoreBefore = null;

addOnUISdk.ready.then(async () => {
  sandbox = await addOnUISdk.instance.runtime.apiProxy("documentSandbox");
  document.getElementById("run").onclick = runPreflight;
  document.getElementById("fixall").onclick = fixAll;
});

async function renderPng() {
  const [r] = await addOnUISdk.app.document.createRenditions(
    { range: addOnUISdk.constants.Range.currentPage, format: addOnUISdk.constants.RenditionFormat.png },
    addOnUISdk.constants.RenditionIntent.preview);
  return r.blob;
}

async function toJpegBase64(blob, maxSide = 1024) {
  const bmp = await createImageBitmap(blob);
  const k = Math.min(1, maxSide / Math.max(bmp.width, bmp.height));
  const cv = new OffscreenCanvas(Math.round(bmp.width * k), Math.round(bmp.height * k));
  cv.getContext("2d").drawImage(bmp, 0, 0, cv.width, cv.height);
```

```
  const buf = new Uint8Array(await (await cv.convertToBlob({ type: "image/jpeg",
      quality: 0.85 })).arrayBuffer());
  let s = ""; for (let i = 0; i < buf.length;
      i += 0x8000) s += String.fromCharCode(...buf.subarray(i, i + 0x8000));
  return btoa(s);
}

function aiActIssues(d) {
  if (scan.texts.some((t) => /generat(ă|a)? cu ai/i.test(t.text))) return [];   // eticheta există deja
  if (d.disclosureRequired) return [{ axis: "ai_act", severity: provenance.aiGenerated ? "error" : "warning",
      title: "Imagine AI fotorealistă fără etichetă", explanation: d.reasoning,
      legalBasis: "EU AI Act, art. 50(4)", fix: { kind: "label" } }];
  if (provenance.aiGenerated) return [{ axis: "ai_act", severity: "info",
      title: "Conținut AI: etichetă recomandată, nu obligatorie", explanation: d.reasoning,
      legalBasis: "Bună practică (Codul de conduită UE)", fix: { kind: "label" } }];
  return [];
}

async function runPreflight() {
  render({ loading: true });
  scan = await sandbox.scanPage();
  const png = await renderPng();
  const det = [...await checkContrast(png, scan), ...checkPalette(scan, KIT), ...checkDisclaimer(scan, KIT)];
  render({ issues: det, aiPending: true });                       // < 1 s: rezultate deterministe
  const res = await fetch(API_AUDIT, { method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ imageBase64: await toJpegBase64(png), texts: scan.texts, brandKit: KIT,
        provenance }) });
  const ai = await res.json();
  if (!res.ok) { render({ issues: det, aiError: ai.error }); return; }
  const llm = ai.issues.map((i) => ({ ...i,
    fix: i.suggestedText && i.nodeId ? { kind: "replaceText", nodeId: i.nodeId,
        text: i.suggestedText } : null }));
  issues = [...det, ...llm, ...aiActIssues(ai.aiDisclosure)];
  const s = score(issues);
  if (scoreBefore === null) scoreBefore = s.total;
  render({ issues, score: s });
}
```

4.2 Randarea (HTML simplu + tokens PreFlight)

addon/src/styles.css

```
:root { --ink:#0F172A; --indigo:#4338CA; --indigo-l:#EEF2FF; --green:#15803D; --green-l:#DCFCE7;
        --amber:#B45309; --amber-f:#F59E0B; --amber-l:#FEF3C7; --red:#B91C1C; --red-l:#FEE2E2;
        --slate:#475569; --border:#E2E8F0; --surface:#F8FAFC; }
body { font: 13px/1.4 "adobe-clean", Inter, system-ui, sans-serif; color: var(--ink); margin: 0; }
.ring { --p: 0; width: 120px; height: 120px; border-radius: 50%; margin: 12px auto; display: grid;
        place-items: center; background: conic-gradient(var(--c) calc(var(--p) * 1%), var(--border) 0); }
.ring > div { width: 96px; height: 96px; border-radius: 50%; background: #fff; display: grid;
    place-items: center;
              font: 700 28px Poppins, sans-serif; }
.card { border-left: 3px solid var(--sev); background: var(--bg); border-radius: 6px; padding: 8px 10px;
        margin: 6px 0; display: grid; grid-template-columns: 1fr auto; gap: 2px 8px; }
.card .sev { font-size: 10px; font-weight: 700; color: var(--sev); } .card .why { color: var(--slate);
    font-size: 11px; }
.error { --sev: var(--red); --bg: var(--red-l); } .warning { --sev: var(--amber); --bg: var(--amber-l); }
.info { --sev: var(--indigo); --bg: var(--indigo-l); } .fixed { opacity: .45; text-decoration: line-through; }
.primary { width: 100%; background: var(--indigo); color: #fff; border: 0; border-radius: 8px; padding: 10px;
    font-weight: 700; }
.skeleton { height: 46px; border-radius: 6px; background: linear-gradient(90deg,#F1F5F9,#E2E8F0,#F1F5F9);
            background-size: 200% 100%; animation: sh 1.2s infinite;
                } @keyframes sh { to { background-position: -200% 0 } }
```

addon/src/ui/render.js (esențialul)

```
function animateScore(to) {                       // numărare 400 ms + culoarea inelului
  const ring = document.getElementById("ring"), num = ring.firstElementChild;
  const from = Number(num.textContent) || 0, t0 = performance.now();
  const col = to >= 85 ? "var(--green)" : to >= 50 ? "var(--amber-f)" : "var(--red)";
  (function step(t) {
    const k = Math.min(1, (t - t0) / 400), v = Math.round(from + (to - from) * k);
    num.textContent = v; ring.style.setProperty("--p", v); ring.style.setProperty("--c", col);
    if (k < 1) requestAnimationFrame(step);
  })(t0);
}
// render({issues, score, aiPending}): golește lista, adaugă câte un .card per problemă
// (titlu, temei, buton "Fix" dacă i.fix există, "Cere preț" dacă i.needsHumanInput),
// 2 x .skeleton dacă aiPending, apoi animateScore(score.total) și actualizează cele 4 chip-uri.
```

Go / No-Go

- ☐ Pe VitaBoost: rezultatele deterministe apar imediat, apoi cardurile AI în câteva secunde.
- ☐ Scorul afișat e **54** (sau aproape; verifică dacă LLM-ul a găsit exact cele 3 probleme legale).
- ☐ Panoul se citește bine la 320 px lățime.

Dacă NU trece: dacă scorul diferă, ajustează prompt-ul cu exemple concrete pentru VitaBoost (few-shot) și verifică cache-ul. Nu modifica formula de scor.

FAZA 5 H+8:00 – 9:30

Fix all + re-verificare

Obiectiv: un click repară tot ce se poate repara în document, iar scorul urcă vizibil (54 → 96) printr-o re-verificare reală.

`addon/src/ui/index.js`

```
async function applyFix(i) {
  const f = i.fix; if (!f || i.fixed) return;
  if (f.kind === "textColor")   await sandbox.setTextColor(f.nodeId, f.hex);
  if (f.kind === "fill")      { await sandbox.setFill(f.nodeId, f.hex);
                                for (const t of f.textFixes) await sandbox.setTextColor(t.nodeId, t.hex); }
  if (f.kind === "replaceText") await sandbox.replaceText(f.nodeId, f.text);
  if (f.kind === "label")       await sandbox.addLabel("Imagine generată cu AI", { corner: "br", size: 22 });
  if (f.kind === "disclaimer")  await sandbox.addLabel(KIT.requiredDisclaimer,
                                     { corner: "bl", size: 14, color: "#334155", bg: null });
  i.fixed = true;
}

async function fixAll() {
  // ordinea contează: întâi textul (rescriere), apoi culoarea aceluiași nod
  const order = ["replaceText", "textColor", "fill", "disclaimer", "label"];
  const todo = issues.filter((i) => i.fix && !i.fixed)
                     .sort((a, b) => order.indexOf(a.fix.kind) - order.indexOf(b.fix.kind));
  for (const i of todo) await applyFix(i);
  render({ issues, score: score(issues) });   // feedback instant (cardurile tăiate, scorul animat)
  await runPreflight();                        // re-verificare REALĂ pe documentul modificat
  logAudit();                                  // Faza 6: jurnal în Firestore
}
```

Pentru demo

- Express are **Undo**: după demo, Ctrl/Cmd+Z de câteva ori readuce asset-ul. Ține totuși și un duplicat al paginii, nemodificat.
- Dacă eticheta AI apare de două ori, re-verificarea nu o detectează. Verifică regex-ul din `aiActIssues` pe textul exact.
- Dacă după re-verificare LLM-ul găsește o problemă nouă în textul rescris, e un argument în plus pentru pitch: „verificarea e reală”. Pentru un demo stabil, rulează Fix all pe asset-ul A de 2–3 ori înainte, ca să încălzești cache-ul.

Go / No-Go

- ☐ Fix all pe VitaBoost: titlul devine închis la culoare și rescris, subtitlul e rescris, CTA-ul devine portocaliu cu text închis, apar disclaimerul și eticheta AI.
- ☐ Scorul final e ~96, iar singura atenționare rămasă e „-70%”, cu butonul „Cere preț”.
- ☐ Tot fluxul (import → PreFlight → Fix all) durează < 15 secunde.

Dacă NU trece: dacă replaceText dă probleme (stiluri pierdute), păstrează doar butonul „Copiază rescrierea” pentru claim-uri și lasă automate culorile, eticheta și disclaimerul. Scorul tot urcă vizibil.

FAZA 6 H+9:30 – 10:30

Asset-uri, jurnal de audit, extra

Obiectiv: demo-ul are tot materialul, iar fiecare verificare lasă o urmă în Firestore. Opționalele intră doar dacă rămâne timp.

6.1 Asset-urile de demo (MUST: 20 min)

- ☐ **A: VitaBoost**, exact ca în dosar (secțiunea 8): 1080×1080, fundal #F3EDE4, titlu #C9C2B8, CTA #E11D48 ca dreptunghi simplu, imaginea Firefly importată **prin panoul PreFlight**. Duplică pagina.
- ☐ **B: Brutăria**: ilustrație stilizată (flat/3D) generată cu Firefly, text corect. Rezultatul așteptat: AI Act „recomandat, nu obligatoriu".
- ☐ **C: Curat**: fără probleme, scor 100.
- ☐ Rulează PreFlight pe A, B, C de câte 2 ori (încălzește cache-ul).

6.2 Jurnal de audit (MUST: 15 min)

backend/functions/index.js (adaugă)

```
exports.logAudit = onRequest({ region: "europe-west1", cors: true }, async (req, res) => {
  const { kit, scoreBefore, scoreAfter, axes, issues, provenance } = req.body || {};
  const ref = await db.collection("audits").add({ kit, scoreBefore, scoreAfter, axes, provenance,
    issues: (issues || []).map(({ axis, severity, title, legalBasis = null, fixed = false }) =>
      ({ axis, severity, title, legalBasis, fixed })),
    createdAt: FieldValue.serverTimestamp() });
  res.json({ id: ref.id });
});
```

addon/src/ui/index.js

```
async function logAudit() {
  const s = score(issues);
  fetch("https://URL-LOGAUDIT", { method: "POST", headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ kit: KIT.name, scoreBefore, scoreAfter: s.total, axes: s.axes, issues,
        provenance }) });
}
```

În demo poți arăta 3 secunde din consola Firestore cu documentul nou din `audits`: „fiecare verificare e o dovadă de diligență".

6.3 Adobe PDF Services: ghid de brand PDF brand kit (SHOULD: 30 min)

- ☐ Pe developer.adobe.com creează credențiale gratuite pentru **PDF Services API** (OAuth Server-to-Server: client ID + secret). Salvează-le ca secrete Firebase.
- ☐ `npm i @adobe/pdfservices-node-sdk adm-zip` în functions.
- ☐ Pornește de la sample-ul oficial **Extract PDF** pentru Node.js, pentru că API-ul SDK-ului se poate schimba între versiuni. Fluxul e cel de mai jos.

backend/functions/brandkit.js (schelet)

```
const { ServicePrincipalCredentials, PDFServices, MimeType, ExtractPDFParams, ExtractElementType,
        ExtractPDFJob, ExtractPDFResult } = require("@adobe/pdfservices-node-sdk");
// 1. credentials = new ServicePrincipalCredentials({ clientId, clientSecret })
// 2. pdfServices = new PDFServices({ credentials })
// 3. inputAsset = await pdfServices.upload({ readStream, mimeType: MimeType.PDF })
// 4. job = new ExtractPDFJob({ inputAsset,
    params: new ExtractPDFParams({ elementsToExtract: [ExtractElementType.TEXT] }) })
// 5. pollingURL = await pdfServices.submit({ job })
// 6. result = await pdfServices.getJobResult({ pollingURL, resultType: ExtractPDFResult })
// 7. stream = await pdfServices.getContent({ asset: result.result.resource })  -> ZIP cu structuredData.json
// 8. textul extras -> LLM cu schemă { palette[], fonts[], tone, requiredDisclaimer } -> Firestore brandKits
```

Pentru demo, fă un ghid de brand VitaBoost de o pagină în Express, exportă-l ca PDF și urcă-l. Arată 10 secunde: „ghidul de brand al clientului devine automat regulile PreFlight".

6.4 Slack (COULD: 10 min)

Creează un Incoming Webhook în Slack și salvează-l ca secret. La sfârșitul `logAudit`, dacă `scoreAfter ≥ 85`, trimite un POST cu `{ text: "VitaBoost aprobat de PreFlight: 96/100" }`.

Oprește-te aici

H+10:30: CODE FREEZE. `git commit -am "freeze" && git tag demo`. De acum înainte repari doar bug-uri care strică demo-ul.

FAZA 7 H+10:30 – 12:00

Pitch, video de backup, repetiții

Obiectiv: o prezentare de ~3 minute fără greșeli, cu un demo care nu poate eșua.

- ☐ **Deck-ul (40 min):** 8 slide-uri după șablonul și scriptul din dosar (secțiunea 9). Culorile și fonturile din secțiunea 6. Slide-ul de arhitectură e diagrama din secțiunea 4.
- ☐ **Video de backup (15 min):** înregistrează ecranul (OBS / QuickTime / Win+G) cu tot fluxul A: import → PreFlight → Fix all → scor 96. Maximum 70 de secunde, fără sunet. Pune-l pe desktop.
- ☐ **Repetiția 1**, cronometrată. Taie tot ce trece de 3 minute.
- ☐ **Repetiția 2** pe Wi-Fi-ul sălii, cu proiectorul dacă se poate. Verifică dacă panoul se vede (zoom în Express).
- ☐ **Repetiția 3** cu întrebări: cineva îți pune 5 întrebări din dosar (secțiunea 11).
- ☐ **Pregătirea scenei:** Express deschis pe asset-ul A nemodificat, add-on conectat, cache încălzit, DevTools închis, notificări oprite, hotspot pregătit, video pe desktop.

Pe scenă

- Deschide cu cifra (3% / 60.000 €), nu cu „Bună ziua, noi suntem echipa…”.
- În timpul demo-ului, vorbește despre **ce vede juriul**, nu despre cod.
- Când scorul urcă la 96, fă o pauză de o secundă. E momentul pe care îl țin minte.
- Termină cu tagline-ul și taci. Nu adăuga „…și cam asta a fost”.

Anexă: depanare rapidă

Simptom	Cauză probabilă	Rezolvare
Add-on-ul nu se încarcă în Express	Certificat local neacceptat / developer mode oprit	Deschide https://localhost:5241 și acceptă certificatul; reactivează Add-on Development; repornește npm run start
apiProxy("documentSandbox") eșuează	Lipsește intrarea documentSandbox din manifest	Folosește template-ul *-with-document-sandbox; verifică manifest.json
Eroare CORS la fetch	Funcția nu are cors: true sau URL greșit	Adaugă cors: true în opțiuni, redeploy, copiază URL-ul exact din output
Mixed content / blocked	Apel http către emulator	Folosește URL-ul https al funcției deployate
Deploy: billing required	Plan Spark	Treci pe Blaze (cu alertă de buget)
Secret not found	Secretul nu e declarat în funcție	functions:secrets:set + secrets: [GEMINI_API_KEY] în opțiunile funcției
scanPage() fără texte	Texte în grupuri / alt nume de tip	Scoate-le din grup; loghează n.type; fallback pe selecție

Simptom	Cauză probabilă	Rezolvare
Contrast greșit / zonă greșită	Bounds relative la grup sau scară greșită	Fără grupuri; verifică sx/sy (lățimea rendition / lățimea paginii)
429 / quota de la LLM	Rate limit în free tier	Cache, retry cu 2 s pauză, model mai mic; activează facturarea
JSON.parse eșuează	Modelul a ieșit din schemă	responseMimeType + responseSchema setate, temperatură 0; loghează resp.text
c2pa: nu găsește manifest	Imaginea a fost re-salvată / exportată	Folosește fișierul descărcat direct din Firefly; verifică pe contentcredentials.org
Fix all aplică eticheta de două ori	Regex-ul nu recunoaște textul	Testează /generat(ă|a)? cu ai/i pe textul exact
Totul pică în timpul pitch-ului	Rețea / Murphy	Video de backup, fără scuze lungi: „vă arăt înregistrarea”

Cum folosești chat-ul ăsta în timpul hackathonului

Când ceva nu merge, lipește în chat: (1) eroarea completă din consolă, (2) fișierul în care apare, (3) ce ai încercat. Lucrăm pe varianta reală a codului tău, nu pe presupuneri.